

M4 — Fundamentos de machine learning y optimización

Puente al SciML: cómo se ajusta un modelo con datos

Proyecto Wilson–Cowan — Iniciación a la I+D ITBA

12 de julio de 2026

Índice

1. Aprendizaje supervisado, parámetros y función de pérdida	1
2. Descenso por gradiente, tasa de aprendizaje y mal condicionamiento	2
3. Optimizadores del proyecto: Adam y L-BFGS	3
4. Sobreajuste, validación y regularización L2	3
5. Diferenciación automática (autograd)	4
6. Trucos de entrenamiento usados en el repo	5

Requisito previo: M1 (álgebra lineal y cálculo vectorial: gradiente, Jacobiano, Hessiano, autovalores, número de condición). Este documento es el **puente** entre esa base y los métodos de SciML que usamos para identificar los 10 parámetros de Wilson–Cowan (WC): la diferenciación automática de la Sección 5 es la maquinaria que reaparece en M5 (residuo físico de la PINN) y en M6 (matriz de sensibilidad J y análisis de Fisher). No es un curso de deep learning: es lo justo para leer y defender el pipeline de entrenamiento del proyecto.

1. Aprendizaje supervisado, parámetros y función de pérdida

Definición 1 (Aprendizaje supervisado). *Tenemos pares de entrada–salida $\{(u_k, y_k^{\text{obs}})\}_{k=1}^N$ y un modelo f_{θ} con **parámetros** $\theta \in \mathbb{R}^p$. Aprender es buscar el θ que hace que las predicciones $\hat{y}_k = f_{\theta}(u_k)$ se parezcan lo más posible a los datos observados.*

Intuición. Un modelo es una familia de funciones indexada por θ ; entrenar es elegir el miembro de esa familia que mejor explica los datos. La cercanía se mide con una **función de pérdida** (o costo) $L(\theta)$. La más común es el error cuadrático medio (MSE):

$$L(\theta) = \frac{1}{N} \sum_{k=1}^N \|y_k^{\text{obs}} - f_{\theta}(u_k)\|^2.$$

Minimizar el MSE equivale a la estimación por máxima verosimilitud cuando el ruido de medición es gaussiano de varianza constante (este vínculo es exactamente el que conecta el MSE con la matriz de Fisher en M6).

Por qué en este proyecto. Nuestro modelo NO es una red genérica: es *gray-box*. La “función” f_{θ} es **integrar las EDOs de Wilson–Cowan** (ver M2) con un conjunto candidato de parámetros y comparar la trayectoria simulada $\hat{y}(t) = \hat{E}(t) - \hat{I}(t)$ con la observada $y^{\text{obs}}(t)$. Los “parámetros” son los 10 físicos de la Tabla canónica ($w_{EE}, w_{EI}, w_{IE}, w_{II}, \tau_e, \tau_i, a_e, a_i, \theta_e, \theta_i$), no pesos abstractos de una red. La pérdida es el MSE entre la salida medida $y = E - I$ y la simulada.

Observación 1. *Los offsets k_e, k_i **no** son parámetros libres: se derivan de las ganancias y umbrales para que el reposo $E = I = 0$ sea equilibrio (ver M2/B3). Reducir el número de grados de libertad libres es una decisión de modelado que ayuda al condicionamiento del problema.*

2. Descenso por gradiente, tasa de aprendizaje y mal condicionamiento

Definición 2 (Descenso por gradiente). *El gradiente $\nabla_{\theta}L$ apunta en la dirección de máximo aumento de la pérdida. El descenso da pasos en la dirección opuesta:*

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta}L(\theta_t),$$

donde $\eta > 0$ es la **tasa de aprendizaje** (learning rate).

La tasa de aprendizaje. Es la perilla más sensible. Si η es muy chica, el entrenamiento se arrastra; si es muy grande, los pasos sobrepasan el mínimo y la pérdida diverge u oscila. No hay un valor universal: depende de la curvatura del paisaje $L(\theta)$.

Mal condicionamiento y zigzag. El problema central aparece cuando el paisaje de la pérdida es un **valle largo y angosto**. Cerca de un mínimo, L se aproxima por un paraboloide cuya forma la fija el Hessiano $H = \nabla^2 L$ (ver M1). Sus autovalores miden la curvatura en cada dirección propia; el **número de condición** $\kappa(H) = \lambda_{\text{máx}}/\lambda_{\text{mín}}$ mide cuán alargado es el valle. Cuando κ es grande:

- una tasa η que sea estable en la dirección *muy curvada* (autovalor grande) es demasiado chica para avanzar en la dirección *plana* (autovalor chico);
- el gradiente apunta casi perpendicular al eje largo del valle, así que el iterado **rebota de pared a pared** (zigzag) y progresa lentísimo hacia el mínimo (Figura 1).

Esto no es un detalle académico: es *el mismo* mal condicionamiento que en M6 diagnosticamos con Fisher+SVD como la razón por la que w_{II} no se identifica. El valle plano de la optimización y el valle plano de la información de Fisher son la misma geometría vista desde el gradiente y desde la curvatura.

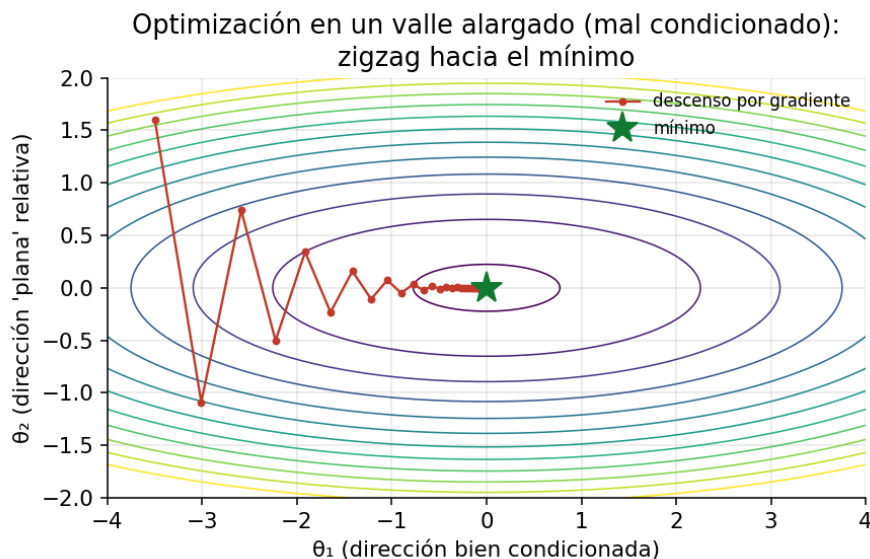


Figura 1: Descenso por gradiente en un valle mal condicionado. La trayectoria zigzaguea entre las paredes empinadas mientras avanza a paso lento por el eje plano del valle. Aumentar η para acelerar el eje plano hace divergir la dirección empinada. (figura del proyecto)

3. Optimizadores del proyecto: Adam y L-BFGS

El descenso por gradiente puro sufre el zigzag de la Sección 2. Los optimizadores reales lo mitigan de dos formas complementarias, y el proyecto usa **las dos, en secuencia**.

Adam (primer orden, robusto). Adam [3] usa solo el gradiente (primer orden) pero adapta el paso por coordenada: mantiene promedios móviles del gradiente (momento, que suaviza el zigzag) y de su cuadrado (para normalizar la escala de cada parámetro). Esto lo vuelve **robusto**: tolera gradientes ruidosos y mal escalados, y avanza bien aun sin ajustar η con precisión quirúrgica. Su debilidad es que, cerca del óptimo, se estanca: da pasos pequeños y ruidosos sin converger fino.

L-BFGS (cuasi-Newton, refina al final). L-BFGS [2] es un método **cuasi-Newton**: aproxima la curvatura (el inverso del Hessiano) a partir del historial reciente de gradientes, sin formar ni invertir H explícitamente (la “L” es de *limited memory*). Al incorporar curvatura, **corrige el zigzag**: en un valle mal condicionado apunta hacia el fondo, no de pared a pared, y converge con precisión de muchas cifras cerca del óptimo. A cambio es más frágil lejos del mínimo (la aproximación de curvatura puede fallar con gradientes ruidosos) y más caro por paso.

Por qué se usan los dos. La estrategia estándar en SciML, y la que seguimos, es una **cascada**:

1. **Adam para el grueso.** Arranca desde una inicialización cualquiera y lleva θ al entorno del óptimo de forma robusta, absorbiendo el ruido y el mal escalado inicial.
2. **L-BFGS para afinar.** Toma el resultado de Adam como punto de partida (ya cerca del mínimo, donde su aproximación de curvatura es válida) y **refina** hasta el fondo del valle con alta precisión.

Es la combinación de un “martillo robusto” seguido de un “destornillador de precisión”. En la identificación de WC esto importa: el fondo del valle de w_{II} es tan plano que Adam solo nunca lo asienta, y L-BFGS solo, arrancando lejos, se desestabiliza.

4. Sobreajuste, validación y regularización L2

Definición 3 (Sobreajuste). *Un modelo **sobreajusta** cuando reduce la pérdida sobre los datos de entrenamiento memorizando su ruido en lugar de capturar la estructura subyacente. Se detecta porque la pérdida de entrenamiento baja pero la de validación (datos no usados para ajustar) empieza a subir.*

Train / validación. Partimos los datos en un conjunto de **entrenamiento** (se usa para mover θ) y uno de **validación** (solo se mira para juzgar generalización). En el proyecto la validación es natural: entrenamos con ciertas trayectorias/estímulos y evaluamos sobre una trayectoria de *test* distinta, típicamente un *chirp* (ver `fit_chirp_test` en los resultados). Que el modelo identificado prediga bien un estímulo que no vio es la prueba de que recuperó la *dinámica* y no un ajuste espurio.

Regularización L2 (mirada inicial). Una forma clásica de frenar el sobreajuste y, sobre todo, de **domesticar el mal condicionamiento** es añadir a la pérdida una penalización cuadrática sobre los parámetros:

$$L_{\text{reg}}(\theta) = \frac{1}{N} \sum_k \|y_k^{\text{obs}} - f_{\theta}(u_k)\|^2 + \lambda \|\theta - \theta_0\|^2.$$

El término $\lambda \|\theta - \theta_0\|^2$ tira de θ hacia un valor de referencia θ_0 (un *prior*), sumando información donde los datos no restringen. Geométricamente, agregar λI a la curvatura **rellena las direcciones planas** del valle y las vuelve estimables. El hiperparámetro λ controla el balance sesgo-varianza: más λ = menos varianza (estimación más estable) a costa de sesgo hacia θ_0 .

Observación 2. *Dejamos aquí solo la intuición. El tratamiento completo — Tikhonov/ridge, interpretación MAP bayesiana, la equivalencia $\lambda \rightarrow \infty$ con “fijar” un parámetro, y el barrido de λ para w_{II} (Exp A2, $\lambda^* \approx 1.0$) — va en **M6 (identificabilidad y OED)**, porque ahí la regularización aparece como remedio directo al valle plano de la matriz de Fisher.*

5. Diferenciación automática (autograd)

Todo lo anterior necesita un ingrediente: el **gradiente** $\nabla_{\theta} L$ (y, en M6, el Jacobiano completo $J = \partial y / \partial \theta$). La **diferenciación automática** (AD, *autograd*) los calcula de forma exacta y eficiente. Es la pieza técnica central de este documento porque es la base de M5 y M6.

Qué NO es. Conviene despejar dos confusiones:

- **No es diferencias finitas.** No aproxima $\partial L / \partial \theta_j \approx (L(\theta_j + h) - L(\theta_j)) / h$. Ese esquema es *inexacto* (error de truncamiento y de redondeo peleando por h) y *caro* (una evaluación del modelo por parámetro). AD da la derivada al nivel de precisión de máquina.
- **No es derivación simbólica.** No manipula fórmulas para producir una expresión cerrada de la derivada (que explotaría en tamaño). AD trabaja *numéricamente*, propagando derivadas junto con los valores.

La idea: cualquier cómputo (incluso integrar una EDO paso a paso) es una composición de operaciones elementales; AD aplica la regla de la cadena a esa composición, acumulando derivadas operación por operación [4].

Dos modos. La regla de la cadena se puede recorrer en dos sentidos, y cuál conviene depende de la forma del problema (muchas entradas vs. muchas salidas):

- **Modo reverse (retropropagación).** Recorre el grafo hacia atrás desde la salida. Calcula el gradiente de **una** salida escalar respecto de *todas* las entradas en (esencialmente) el costo de una evaluación del modelo. Es **barato cuando hay muchas entradas y una sola salida**: exactamente el caso de la pérdida L (millones de parámetros posibles $\rightarrow$ un número). Es lo que usamos para $\nabla_{\theta} L$ al entrenar.
- **Modo forward (jacfwd).** Recorre el grafo hacia adelante, propagando la derivada respecto de una entrada a través de todo el cómputo. Da una *columna* del Jacobiano por pasada, así que es **eficiente cuando $\# \text{parámetros} \leq \# \text{salidas}$** . Es el modo idóneo para construir la matriz de sensibilidad $J = \partial y / \partial \theta$: acá la “salida” es la trayectoria entera $y(t)$ (muchos puntos) y las “entradas” son solo 10 parámetros.

Por qué en este proyecto (la base de M5 y M6).

- **M5 — residuo físico (PINN).** La PINN penaliza que su predicción viole las EDOs de WC. Para armar ese residuo hay que derivar la salida de la red respecto del *tiempo* (y de sus entradas); eso es AD. Sin diferencias finitas: el residuo físico se evalúa exacto.
- **M6 — sensibilidad y Fisher.** La matriz $J = \partial y / \partial \theta$ se calcula por AD en **modo forward (jacfwd)**, aprovechando que hay solo 10 parámetros contra una trayectoria larga. De J salen la matriz de Fisher $J^T J / \sigma^2$, su SVD y el diagnóstico de identificabilidad. Que J sea *exacta* (no una aproximación por diferencias finitas) es lo que hace confiable el análisis del cuello de botella w_{II} .

Además, entrenar el Neural ODE requiere **backpropagar a través del integrador**: como RK4 (ver M2) es una composición de operaciones diferenciables, AD atraviesa todos los pasos de integración sin que tengamos que escribir a mano ninguna derivada.

6. Trucos de entrenamiento usados en el repo

Ajustar un Neural ODE *gray-box* sobre un sistema oscilante y mal condicionado no funciona con un entrenamiento ingenuo. El repo usa tres trucos que vale la pena conocer:

- **Warmup “datos primero”.** En vez de castigar desde el arranque tanto el ajuste a datos como el residuo físico (o el término de dinámica), se *empieza* priorizando el ajuste a los datos y recién después se sube el peso del término físico. Motivo: al principio los parámetros están lejos y el residuo físico produce gradientes enormes y engañosos; dejar que los datos lleven θ a una zona razonable primero estabiliza todo lo demás.
- **Congelar la red.** En un modelo con dos bloques — la parte física con los 10 parámetros y la parte de red neuronal (features/corrección) — conviene por etapas **congelar** (fijar, sin gradiente) uno mientras se ajusta el otro. Típicamente se fija la red para que los datos determinen primero los parámetros físicos, evitando que la red “tape” errores de los parámetros absorbiéndolos ella (lo que arruinaría la identificación).
- **Parada por meseta.** En vez de un número fijo de épocas, se monitorea la pérdida de validación y se **corta cuando deja de mejorar** (se amesetó) durante un número de épocas dado. Esto es *early stopping*: previene el sobreajuste de la Sección 4 y ahorra cómputo. Encaja con la cascada de la Sección 3: se deja que Adam llegue a la meseta y ahí se pasa el control a L-BFGS para el afinado final.

Observación 3. *Estos trucos son coherentes con toda la lógica del proyecto: el objetivo no es minimizar la pérdida a cualquier precio, sino **recuperar parámetros físicos identificables**. Por eso se cuida que sean los datos — y no la flexibilidad de la red — los que fijen θ .*

Referencias

- [1] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.
- [2] J. Nocedal y S. J. Wright, *Numerical Optimization*, 2^a ed., Springer, 2006.
- [3] D. P. Kingma y J. Ba, “Adam: A Method for Stochastic Optimization”, *ICLR*, 2015.
- [4] A. G. Baydin, B. A. Pearlmutter, A. A. Radul y J. M. Siskind, “Automatic differentiation in machine learning: a survey”, *Journal of Machine Learning Research* 18(153):1–43, 2018.